Measuring Inductance

JQI Schine Lab

April 18, 2025

1 Introduction

This report details the procedure and results from an experimental investigation to measure the inductance of a coil constructed from 10 AWG copper wire with approximately 120 turns. Multiple measurement methodologies were employed to compare their accuracy and reliability. The primary aim was to determine the most effective technique for accurately measuring inductance while identifying potential sources of measurement error.

2 Measurements

The inductance of the coil was measured using four different scenarios. Table 1 summarizes the results obtained through these different methodologies. A single measurement of the DC resistance yielded a value of $0.29\,\Omega$.

Table 1: Inductance Measurements Using Different Methodologies

Scenario	**Method**	**Measured Inductance**
Scenario 0	Oscilloscope pulse decay measurement	3.80 mH
Scenario 1	RLC circuit with resonance peak analysis	2.85 mH
Scenario 2	LC circuit with resonance peak analysis	4.33 mH
Scenario 3	LC circuit with impedance slope method	4.83 mH

3 Brief Overview

A Bode Plot 100 analyzer was utilized to conduct impedance measurements of both RLC and LC circuit configurations to determine the inductance of the coil. The measurements were performed with three different configurations: with a $1\,\mu F$ capacitor (C1), with a 100 nF capacitor (C2), and without any capacitor. Although the Bode plotter features three channels, only the output channel was employed for this experiment, while channels 1 and 2 remained unused.

Figure 1: Circuit Diagram for RLC Configuration

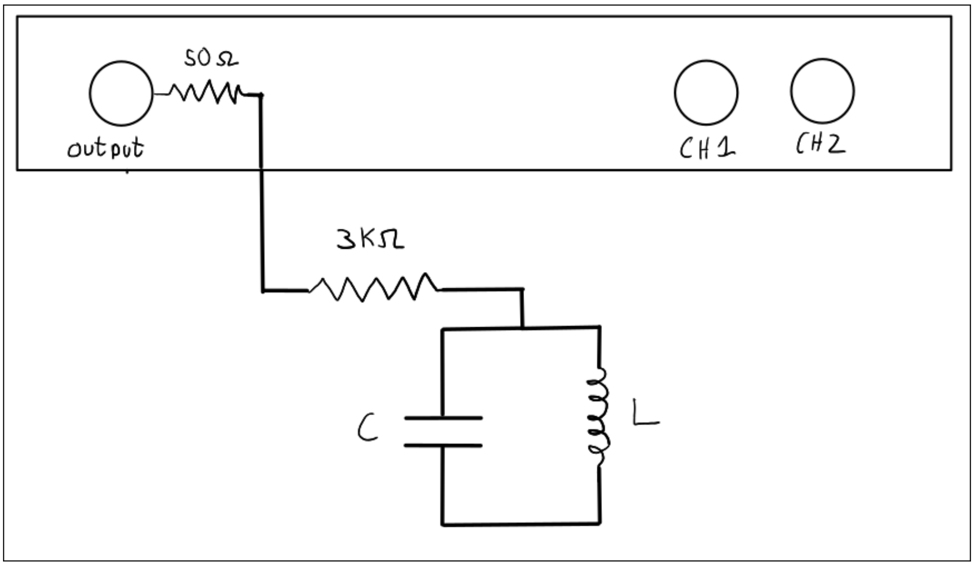
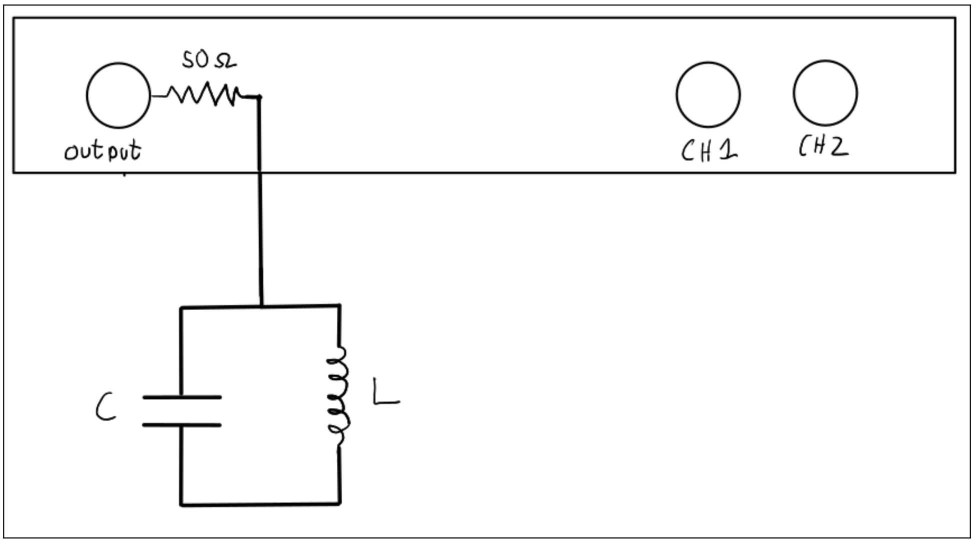


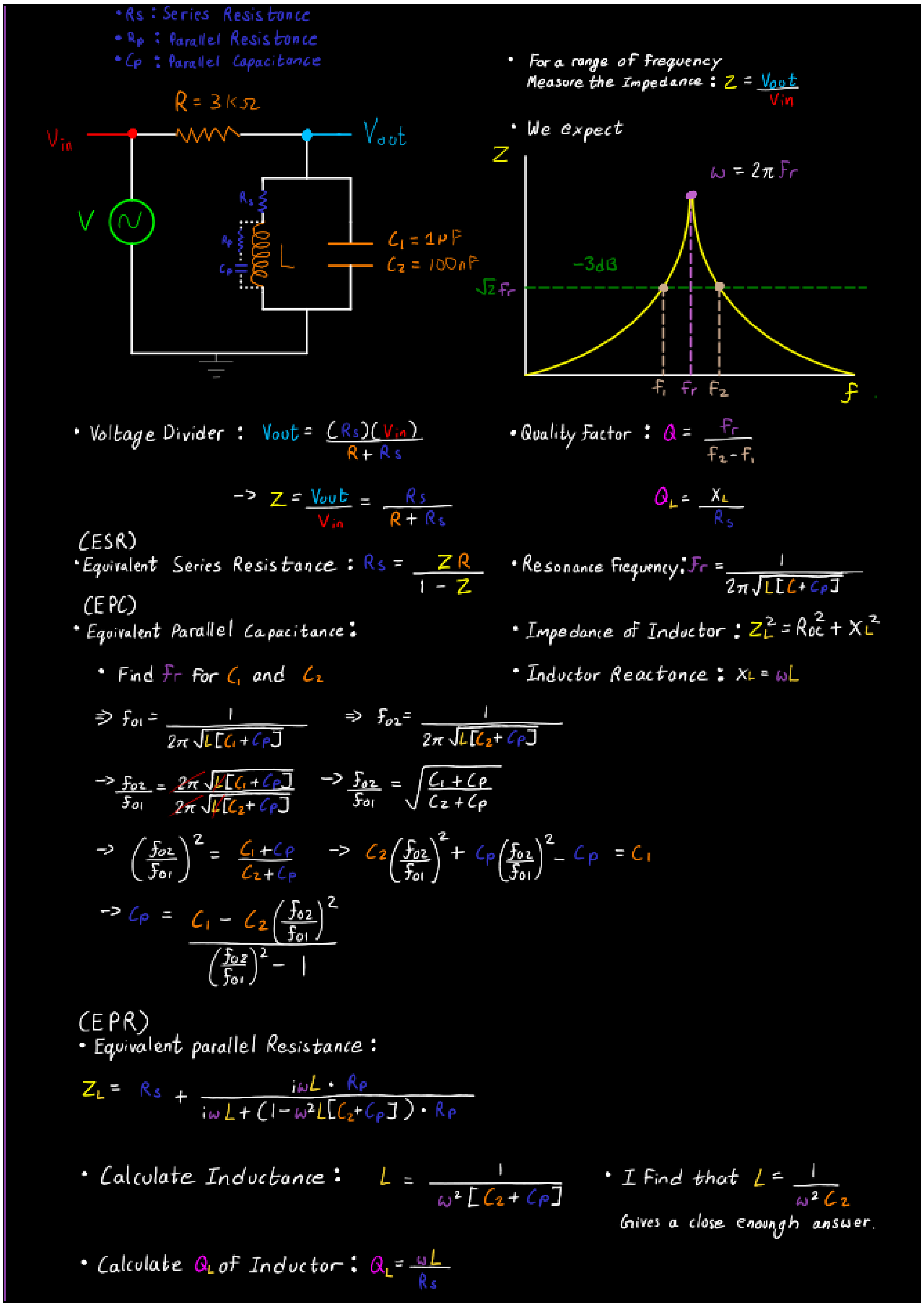
Figure 2: Circuit Diagram for LC Configuration



Since the Bode plotter software refused to connect to my Windows system, I opted to use a Linux environment. An SCPI (Standard Commands for Programmable Instruments) server was used to communicate with the Bode Plot 100 analyzer. Please refer to the automation section in https://www.omicron-lab.com/downloads/bode-analyzer-suite for more details. While the windows software provides a graphical application, the automated data collection enabled precise frequency sweeps and consistent measurement settings across all test configurations.

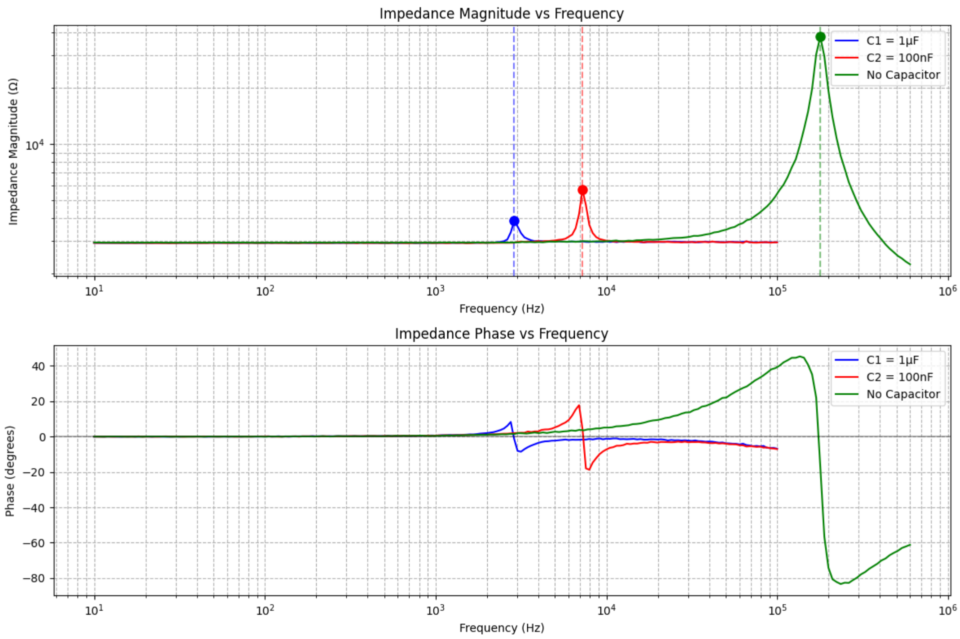
4 Initial Planning

Figure 3: Hand-Written Experimental Notes and Planning



The initial experimental approach utilized an RLC circuit with a 3 kΩ resistor to measure the inductance of the coil. The impedance characteristics were measured across a frequency range from 10 Hz to 1000 kHz.

Figure 4: Impedance vs. Frequency Plot for RLC Circuit (Log-Log Scale)

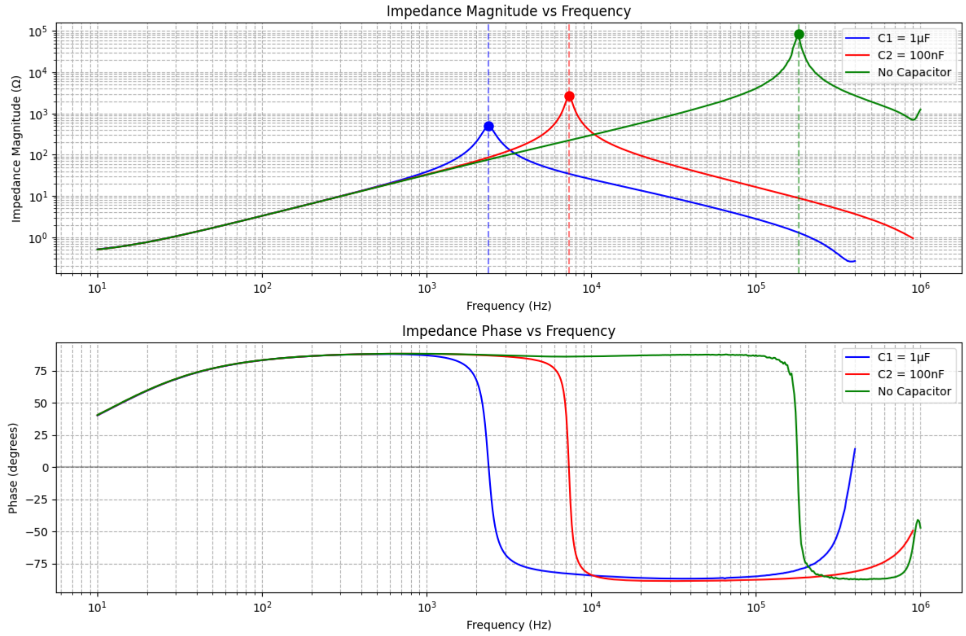


Analysis of the RLC circuit data revealed that the resistive component significantly damped the system's energy, obscuring the expected linear slope in the impedance versus frequency plot that is characteristic of inductive components. Instead, the resonance peaks observed in the frequency response were used to calculate the inductance as detailed in the hand-written notes (Figure 3).

However, it became apparent that the Bode Plotter was measuring the impedance of the entire RLC circuit, not just the inductor. This realization indicated that the calculation methodology outlined in the hand-written notes would be applied incorrectly in this scenario, as it assumes that the impedance-frequency relationship is solely attributed to the inductive component. This discovery prompted a shift to an LC circuit configuration for subsequent measurements.

5 Slope Method

Figure 5: Impedance vs. Frequency Plot for LC Circuit (Log-Log Scale)



The LC circuit configuration provided clearer measurement data, with all three test configurations (C1, C2, and no capacitor) exhibiting the expected linear slope in the log-log plot of impedance versus frequency. This linear relationship in the logarithmic domain confirms the dominant inductive behavior in the frequency range of interest.

For the inductance calculation, the impedance data from the "no capacitor" configuration was used, as this represents the purest measurement of the inductor's characteristics without the influence of capacitive elements. The slope of this line corresponds directly to the inductor's impedance relationship:

$$Z_L = j\omega L = j2\pi f L \tag{1}$$

Where the magnitude of impedance increases linearly with frequency:

$$|Z_L| = 2\pi f L \tag{2}$$

Thus, the inductance can be calculated from the slope of the impedance versus frequency plot:

$$L = \frac{\text{slope}}{2\pi} \tag{3}$$

This calculation yielded an inductance value of 4.83 mH, which is considered the most accurate measurement from this experimental investigation.

6 Rough Analysis

A theoretical estimation of the inductance can be calculated using the physical dimensions of the coil:

- Relative permeability (μ_r) = 1 (air core)
- Number of turns (N) = 120
- Outer diameter = 0.2 m
- Inner diameter = 0.14 m
- Depth = 0.04 m

For a coil with these dimensions, the theoretical inductance can be calculated using the formula for a toroidal coil with rectangular cross-section:

$$L = \frac{\mu_0 \mu_r N^2 A}{l} \tag{4}$$

Where:

- μ_0 is the permeability of free space $4\pi \times 10^{-7}$
- A is the cross-sectional area of the coil winding
- l is the mean path length of the magnetic flux

Calculating these parameters:

$$\text{Mean diameter} = \frac{0.2 + 0.14}{2} = 0.17\,\text{m} \tag{5}$$

$$\text{Cross-sectional area} = \frac{0.2 - 0.14}{2} \times 0.04 = 0.0012\,\text{m}^2 \tag{6}$$

$$\text{Mean path length} = \pi \times 0.17 = 0.534\,\text{m} \tag{7}$$

Substituting these values:

$$
\begin{aligned}
L &= \frac{4\pi \times 10^{-7} \times 1 \times 120^2 \times 0.0012}{0.534} && (8)\\
&= \frac{4\pi \times 10^{-7} \times 14400 \times 0.0012}{0.534} && (9)\\
&= \frac{4\pi \times 10^{-7} \times 17.28}{0.534} && (10)\\
&\approx 4.07\,\text{mH} && (11)
\end{aligned}
$$

Comparing this theoretical value with our experimental measurements:

- Scenario 0 (Oscilloscope): 3.80 mH (approximately 6.6% below theory)
- Scenario 1 (RLC circuit): 2.85 mH (approximately 30.0% below theory)
- Scenario 2 (LC resonance): 4.33 mH (approximately 6.4% above theory)
- Scenario 3 (LC slope): 4.83 mH (approximately 18.7% above theory)

The measurements from Scenarios 0 and 2 most closely align with the theoretical calculation, with deviations of less than 7%. The significant deviation in Scenario 1 (RLC circuit) may confirm the earlier observation that the resistor component affects the measurement.

7 Code

The following code was implemented in a Jupyter notebook for automated data collection and analysis:

7.1 Block 1: SCPI Server Setup

```python
# Block 1: SCPI Server Setup
import pyvisa
import time
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit

# *** TO DO: adapt IP address and SCPI port according to your needs ***
SCPI_server_IP = 'Server IP given by SCPI server'
SCPI_Port = '5025'
SCPI_timeout = 20000 # milliseconds
VISA_resource_name = 'TCPIP::' + SCPI_server_IP + '::' + SCPI_Port + '::SOCKET'

# Measurement configuration
Start_frequency = '10Hz'
Stop_frequency = '1000KHz'
Number_of_measurement_points = 601 # the minimum amount of measurement points is 2!
Sweep_type = 'LOG'
Receiver_bandwidth = '300Hz'

def perform_measurement():
    start_time = time.time()
    print(
        'Trying to connect to VISA resource: ' + VISA_resource_name +
        '. Be sure that IP address and port number are correct!')
    visaSession = pyvisa.ResourceManager().open_resource(VISA_resource_name)
    visaSession.timeout = SCPI_timeout
    visaSession.read_termination = '\n'
    print('SCPI client connected to SCPI server: ' + visaSession.query("*IDN?"))

    lockOk = visaSession.query(":SYST:LOCK:REQ?")
    if lockOk != "1":
        print("Locking was not successful, exiting!")
        return None, None, None
    else:
        print("Locking was successful!")

    try:
        ### Here comes the measurement configuration data for measurement sweep
        visaSession.write(':SENS:FREQ:STAR ' + Start_frequency)
        visaSession.write(':SENS:FREQ:STOP ' + str(Stop_frequency))
        visaSession.write(':SENS:SWE:POIN ' + str(Number_of_measurement_points))
        visaSession.write(':SENS:SWE:TYPE ' + Sweep_type)
        visaSession.write(':SENS:BAND ' + Receiver_bandwidth)
```

```
        # configuring 'one-port' impedance measurement
        visaSession.write(':CALC:PAR:DEF Z')
        # linear magnitude in Ohms + phase(deg)
        visaSession.write(":CALC:FORM SLIN")

        # Intializes trigger system to use BUS as source
        visaSession.write(':TRIG:SOUR BUS')
        # Sets the trigger in continous mode.
        visaSession.write(':INIT:CONT ON')
        visaSession.write(':TRIG:SING')

        # This command waits for all pending operations to finish
        visaSession.query('*OPC?')

        # Query the measurement results
        allResults = visaSession.query(":CALC:DATA:SDAT?")
        frequencyValues = visaSession.query(":SENS:FREQ:DATA?")

        # Magnitude and phase from the results
        allResults_list_raw = list(map(float, allResults.split(",")))
        magnitude_raw = allResults_list_raw[0:Number_of_measurement_points]
        phase_raw = allResults_list_raw[Number_of_measurement_points:len(allResults_list_raw)]

        # Same procedure with frequency raw data to list and eventually to array
        freq_list_raw = list(map(float, frequencyValues.split(",")))

        # Printing just the first results of the measurement cycle as example
        print("Frequency: " + str(freq_list_raw[0]) + " Hz;" + "\tMagnitude: "
            + str(magnitude_raw[0])
            + "   ;" + "\tPhase: " + str(phase_raw[0]) + "   ")

    except Exception as e:
        print(f"An error occurred: {e}")
        return None, None, None

    finally:
        # Release the device
        relOK = visaSession.query(':SYSTem:LOCK:RELease?')
        visaSession.query('*OPC?')
        if relOK != "1":
            print("Releasing was not successful, check the availability of the device!")
        else:
            print("Releasing was successful!")

        visaSession.close()
        print("---Measurement time: %s seconds ---" % (time.time() - start_time))

    return freq_list_raw, magnitude_raw, phase_raw
```

Listing 1: SCPI Server Setup and Connection

7.2 Block 2: First Measurement with C1 = 1F

```
# Block 2: First Measurement with C1 = 1 F
print("Running measurement with C1 = 1 F ")
freq1, mag1, phase1 = perform_measurement()
#save to csv file
np.savetxt("1uF.csv", np.column_stack((freq1, mag1, phase1)), delimiter=
    ",", header="Frequency, Magnitude, Phase", comments='')
```

Listing 2: First Measurement with C1

7.3 Block 3: Second Measurement with C2 = 100nF

```
# Block 3: Second Measurement with C2 = 100nF
print("\nRunning measurement with C2 = 100nF")
freq2, mag2, phase2 = perform_measurement()
# save to csv file
np.savetxt("100nF.csv", np.column_stack((freq2, mag2, phase2)),
    delimiter=",", header="Frequency, Magnitude, Phase", comments='')
```

Listing 3: Second Measurement with C2

7.4 Block 4: Third Measurement with No Capacitor

```
# Block 3.1: Third Measurement with no capacitor
print("\nRunning measurement with no capacitor")
freq3, mag3, phase3 = perform_measurement()
#Save to csv file
np.savetxt("NoCap.csv", np.column_stack((freq3, mag3, phase3)),
    delimiter=",", header="Frequency, Magnitude, Phase", comments='')
plt.figure()
plt.loglog(freq3, mag3)
plt.show()
```

Listing 4: Third Measurement with No Capacitor

7.5 Block 5: Analysis and Plotting

```
# Block 4: Analysis and Plotting
import numpy as np
import matplotlib.pyplot as plt

fre1 = np.loadtxt("1uF.csv", delimiter=",", skiprows=1)
fre2 = np.loadtxt("100nF.csv", delimiter=",", skiprows=1)
fre3 = np.loadtxt("NoCap.csv", delimiter=",", skiprows=1)
freq1 = [n for n in fre1[:,0]]
mag1 = [n for n in fre1[:,1]]
phase1 = [n for n in fre1[:,2]]
freq2 = [n for n in fre2[:,0]]
mag2 = [n for n in fre2[:,1]]
phase2 = [n for n in fre2[:,2]]
freq3 = [n for n in fre3[:,0]]
mag3 = [n for n in fre3[:,1]]
phase3 = [n for n in fre3[:,2] ]

```

```
# Find resonant frequencies by finding MAXIMUM impedance magnitude (for parallel resonance)
    # Find maximum impedance (peak in the bode plot)
max_idx1 = mag1.index(max(mag1))
fr1 = freq1[max_idx1]
z_max1 = mag1[max_idx1]
print(f"Resonant frequency (C1): {fr1:.2f} Hz, Maximum impedance: {z_max1:.2f}   ")

# Find maximum impedance (peak in the bode plot)
max_idx2 = mag2.index(max(mag2))
fr2 = freq2[max_idx2]
z_max2 = mag2[max_idx2]
print(f"Resonant frequency (C2): {fr2:.2f} Hz, Maximum impedance: {z_max2:.2f}   ")

# Find maximum impedance (peak in the bode plot)
max_idx3 = mag3.index(max(mag3))
fr3 = freq3[max_idx3]
z_max3 = mag3[max_idx3]
print(f"Resonant frequency (No Capacitor): {fr3:.2f} Hz, Maximum impedance: {z_max3:.2f}   ")


'''
Invalid way of calculating the EPC and L

# Calculate component values if resonant frequencies were found
if fr1 and fr2:
    # Given values
    C1 = 1e-6  # 1 F
    C2 = 100e-9  # 100nF

    # Calculate inductance using equation
    # fr = 1/(2     (L[C+Cp]))
    # Two measurements with different capacitors allow us to solve for L

    # Using the formula derived
    # (fr2/fr1)^2 = (C1+Cp)/(C2+Cp)
    # Solve for Cp (Equivalent Parallel Capacitance)
    ratio_sq = (fr2/fr1)**2
    Cp = (C1 - ratio_sq * C2) / (ratio_sq - 1)
    print(f"Equivalent Parallel Capacitance (Cp): {Cp*1e9:.2f} nF")

    # Calculate inductance using fr1 and total capacitance (C1 + Cp)
    L = 1 / ((2 * np.pi * fr1)**2 * (C1 + Cp))
    print(f"Inductance (L): {L*1e3:.2f} mH")

    # Calculate Slope of the plot at 100Hz to 300Hz by fitting to a line


if fr3:
    L_wc = 1 / (2 * np.pi * fr3)
    print(f"Inductance without cap (L_wc): {L*1e3:.2f} mH")

    # using fr = 1/(2     (LC)) where Cp = C
    Cp_wc = 1 / (4 * (np.pi**2) * (fr3**2) * L_wc)
    print(f"Parasitic Capacitance (Cp_wc): {Cp_wc*1e9:.2f} nF")
```

```
'''
# Use the slope of the plot to calculate the inductance
def calculate_inductance_from_slope(freq, mag, freq_start, freq_end):
    # Find indices for the frequency range
    indices = [i for i, f in enumerate(freq) if freq_start <= f <= freq_end]

    # Extract the relevant portions of the data
    freq_range = [freq[i] for i in indices]
    mag_range = [mag[i] for i in indices]

    # Fit a line to the data
    coeffs = np.polyfit(freq_range, mag_range, 1)
    slope = coeffs[0]
    intercept = coeffs[1]

    # For an inductor, |Z| =  L  = 2 fL
    # L = slope / (2  )
    L = slope / (2 * np.pi)

    return L, slope, intercept, freq_range, mag_range

# Set frequency range for analysis
freq_start = 10**2  # Hz
freq_end = 10**4    # Hz

# Choose which dataset to use ('C1', 'C2', or 'noCapacitor')
dataset = 'noCapacitor'  # Change this to use different dataset

# Calculate inductance based on chosen dataset
if dataset == 'C1':
    L, slope, intercept, freq_range, mag_range = calculate_inductance_from_slope(
        freq1, mag1, freq_start, freq_end)
    label = "C1 = 1 F "
elif dataset == 'C2':
    L, slope, intercept, freq_range, mag_range = calculate_inductance_from_slope(
        freq2, mag2, freq_start, freq_end)
    label = "C2 = 100nF"
elif dataset == 'noCapacitor':
    L, slope, intercept, freq_range, mag_range = calculate_inductance_from_slope(
        freq3, mag3, freq_start, freq_end)
    label = "No Capacitor"

# Print results
if L is not None:
    print(f"\nInductance calculation using {label} (slope between {freq_start}-{freq_end} Hz):")
    print(f"  Linear fit: |Z| = {slope:.4f}f + {intercept:.4f}")
    print(f"  Slope: {slope:.4f}   /Hz")
    print(f"  Inductance: {L*1e3:.2f} mH")
else:
    print(f"Could not calculate inductance for {label} in range {freq_start}-{freq_end} Hz")

# w = 2 f  measured DC Resistance = mdr # Ohm
```

```python
mdr = 0.29
w1 = 2 * np.pi * fr1
w2 = 2 * np.pi * fr2
w3 = 2 * np.pi * fr3
Q1 = w1*L/mdr
print("Q factor with C1 = 1 F : ", Q1)
Q2 = w2*L/mdr
print("Q factor with C2 = 100nF: ", Q2)
Q3 = w3*L/mdr
print("Q factor with no capacitor: ", Q3)

# Plot the results
plt.figure(figsize=(12, 8))

# Plot 1: Magnitude vs Frequency
plt.subplot(2, 1, 1)
if freq1 and mag1:
    plt.loglog(freq1, mag1, 'b-', label='C1 = 1 F ')
    plt.axvline(x=fr1, color='b', linestyle='--', alpha=0.5)
    plt.plot(fr1, z_max1, 'bo', markersize=8)


if freq2 and mag2:
    plt.loglog(freq2, mag2, 'r-', label='C2 = 100nF')
    plt.axvline(x=fr2, color='r', linestyle='--', alpha=0.5)
    plt.plot(fr2, z_max2, 'ro', markersize=8)

if freq3 and mag3:
    plt.loglog(freq3, mag3, 'g-', label='No Capacitor')
    plt.axvline(x=fr3, color='g', linestyle='--', alpha=0.5)
    plt.plot(fr3, z_max3, 'go', markersize=8)


plt.grid(True, which="both", ls="--")
plt.xlabel('Frequency (Hz)')
plt.ylabel('Impedance Magnitude (  )')
plt.title('Impedance Magnitude vs Frequency')
plt.legend()

# Plot 2: Phase vs Frequency
plt.subplot(2, 1, 2)
if freq1 and phase1:
    plt.semilogx(freq1, phase1, 'b-', label='C1 = 1 F ')
if freq2 and phase2:
    plt.semilogx(freq2, phase2, 'r-', label='C2 = 100nF')
if freq3 and phase3:
    plt.semilogx(freq3, phase3, 'g-', label='No Capacitor')
plt.grid(True, which="both", ls="--")
plt.xlabel('Frequency (Hz)')
plt.ylabel('Phase (degrees)')
plt.title('Impedance Phase vs Frequency')
plt.axhline(y=0, color='k', linestyle='-', alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

Listing 5: Analysis and Plotting